



PENSAMENTO COMPUTACIONAL • AULA 14 DE 15

Complexidade de Algoritmos

Como o trabalho de um algoritmo cresce, sentido na prática com atividades desplugadas — roteiro visual para aula ou minicurso.

Objetivos de aprendizagem

- 1 Construir a função de custo $T(n)$ a partir da contagem de passos.
- 2 Interpretar sequência, decisão, repetição e recursão como contribuições distintas ao custo.
- 3 Perceber a diferença entre um algoritmo rápido na prática e um que escala melhor conforme os dados crescem.
- 4 Sentir o crescimento na prática, com atividades desplugadas e exemplos concretos.

A pergunta central

- Um algoritmo que funciona para 100 entradas ainda será viável para 100 mil?
- A análise de complexidade não pergunta apenas “funciona?”. Ela pergunta “como o trabalho cresce?”.
- Essa pergunta antecede o tempo em segundos, porque segundos mudam com a máquina; passos pertencem ao algoritmo.

passos ↑↑ **em função de** **n** ↑

Passos, não segundos

Tempo medido

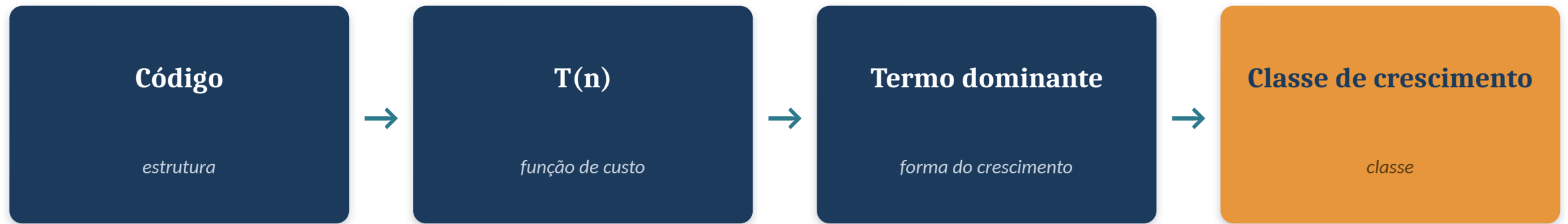
- Depende de processador, cache, compilador, linguagem, SO, I/O e concorrência.

Passos lógicos

- Pertencem ao algoritmo e permitem comparar soluções antes de executar.

Complexidade começa na contagem de ações fundamentais.

Do código ao crescimento



A classe de crescimento é o último passo. Ela não substitui a função de custo.

Ação fundamental

- Antes de contar, é preciso definir o que será contado.
- comparação $\rightarrow v[i] > 0$
- soma $\rightarrow s += v[i]$
- chamada $\rightarrow \text{fib}(n-1)$
- A escolha precisa ser coerente durante toda a análise.

O tamanho da pilha não importa

Atividade desplugada — sinta o que é um custo que não cresce.

1

Material: uma pilha de livros ou cadernos (pequena ou grande).

2

Peça a um aluno para pegar o livro do topo e cronometre. Depois peça para pegar um do meio.

3

Agora dobre o tamanho da pilha e repita: pegar o do topo continua levando o mesmo tempo.

4

Pegar o do topo é sempre 1 passo, não importa quantos livros existam por baixo — isso é custo fixo.

Cada aluno, os mesmos passos

Atividade desplugada — veja o total crescer junto com a fila.

1

Material: nenhum. Simule uma fila de lanche com a turma.

2

Cada aluno faz sempre os 3 mesmos passos: pega a bandeja, escolhe o item, paga.

3

Cronometre 5 alunos na fila, depois 10 — o tempo total dobra junto com o número de alunos.

4

O total cresce proporcional ao número de alunos: dobrar a fila dobra o tempo.

CONSTANTES

Sequências calibram o crescimento

- As três ações dentro do laço geram uma constante multiplicativa.
- $T(n) = 3n$
- O total ainda cresce proporcional a n , mas a constante representa trabalho real.

INSTANCIANDO $T(n)$

n	5	10	20
$T(n) = 3n$	15	30	60

$$T(n) = an + b$$

Constante aditiva e multiplicativa

- Antes do laço: custo fixo.
- Dentro do laço: custo por elemento.
- Depois do laço: custo fixo.
- Exemplo aproximado: $T(n) = 2n + 4$

INSTANCIANDO $T(n)$

n	5	10	20
$T(n) = 2n + 4$	14	24	44

Decidir não muda quantos passos

Atividade desplugada — o caminho muda, o número de passos não.

1

Material: um monte de roupas (ou cartões coloridos representando roupas).

2

Para cada peça, decida em voz alta “lavar” ou “guardar” e coloque na pilha certa.

3

Cronometre 10 peças: decidir e colocar leva praticamente o mesmo tempo por peça, qualquer que seja a decisão.

4

O total ainda cresce proporcional ao número de peças — o “se” não faz o trabalho dobrar.

Condicionais ajustam caminhos

- Mesmo com dois caminhos, apenas um é executado.
- O custo costuma permanecer o mesmo, não importa o tamanho da entrada.
- Mas condicionais importam para melhor caso, pior caso e caso médio.

ONDE O CUSTO SE REPETE

O IF dentro do laço

- O IF é constante, mas está sendo executado n vezes.
- $T(n) \approx c \cdot n$
- A pergunta prática é: quantas vezes esta decisão será tomada?

INSTANCIANDO $T(n)$

n	5	10	20
$T(n) \approx n$ (supondo $c = 1$)	5	10	20

Procurando um por um

Atividade desplugada — sinta o custo de procurar sem atalho.

1

Material: uma lista de nomes da turma, fora de ordem.

2

Escolha um nome e peça para um aluno encontrá-lo, dizendo cada nome em voz alta até achar.

3

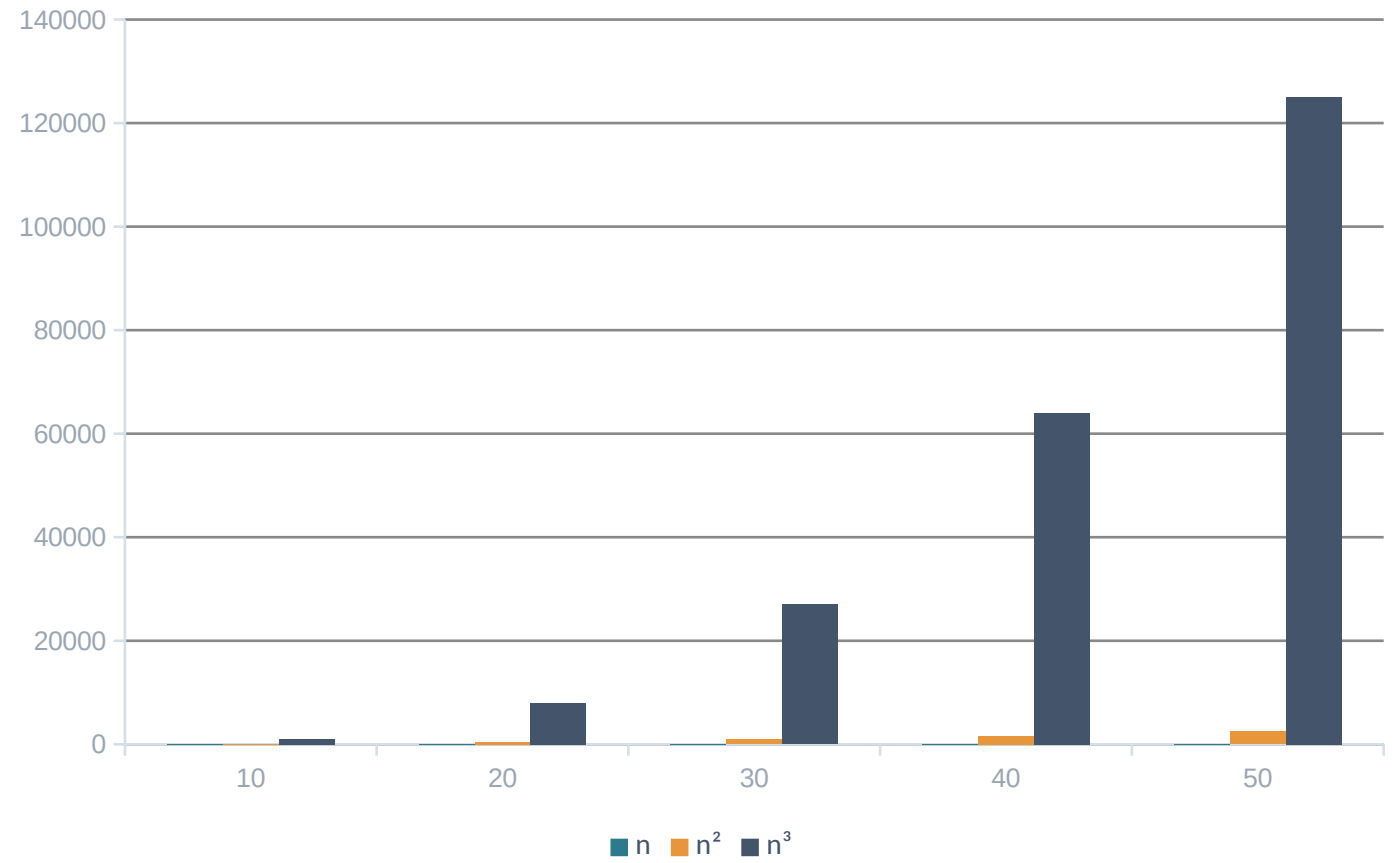
Repita com um nome no início da lista e outro no fim — o segundo demora muito mais.

4

Sem uma lista organizada, encontrar um nome pode exigir olhar a lista inteira.

Laços multiplicam trabalho

- Cada laço aninhado introduz uma nova dimensão de crescimento.
- 1 laço → cresce proporcional a n
- 2 laços → cresce proporcional a n^2
- 3 laços → cresce proporcional a n^3



EXEMPLO

Laço simples: crescimento linear

- Se há uma ação principal por elemento: $T(n) = n$
- Custo marginal: $T(n) - T(n-1) = 1$

INSTANCIANDO $T(n)$

n	5	10	20
$T(n) = n$	5	10	20
Custo marginal: $T(n) - T(n-1) = 1$	1	1	1

Cortando a busca ao meio

Atividade desplugada — descubra como cortar pela metade economiza passos.

1

Material: nenhum. Pense em um número entre 1 e 100 e não conte a ninguém.

2

Os alunos só podem perguntar “é maior ou menor que X?” — cada resposta corta a lista de possibilidades ao meio.

3

Conte quantas perguntas foram precisas para acertar (normalmente, 7 ou menos).

4

Cortar pela metade a cada passo faz a busca crescer muito mais devagar do que ir um por um.

Cortar pela metade economiza passos

- Na lista de chamada, cada tentativa elimina só 1 nome — o trabalho cresce junto com o tamanho da lista.
- No jogo de adivinhar, cada pergunta elimina metade das possibilidades — sobra muito menos para procurar a cada rodada.
- Por isso 100 possibilidades caem para 1 em cerca de 7 perguntas, não em 100 tentativas.

Aperto de mãos: sentindo o n^2

Atividade desplugada — sem computador, só a turma de pé.

1

Material: nenhum. Só a turma, em círculo.

2

Cada aluno aperta a mão de todos os colegas, uma vez cada — contem em voz alta o total ao final.

3

Com n alunos, o total é $n \cdot (n-1)/2$ — a mesma forma de um laço duplo percorrendo todos os pares.

4

Dobrem o grupo (chamem outra sala) e recontem: o número de apertos não dobra, quadruplica.

Laço duplo: crescimento quadrático

- O laço interno executa n vezes para cada valor de i .
- $n \cdot n = n^2$
- Custo marginal aproximado cresce com n .
- E se fossem três laços aninhados? O crescimento fica ainda mais rápido — proporcional a n^3 .

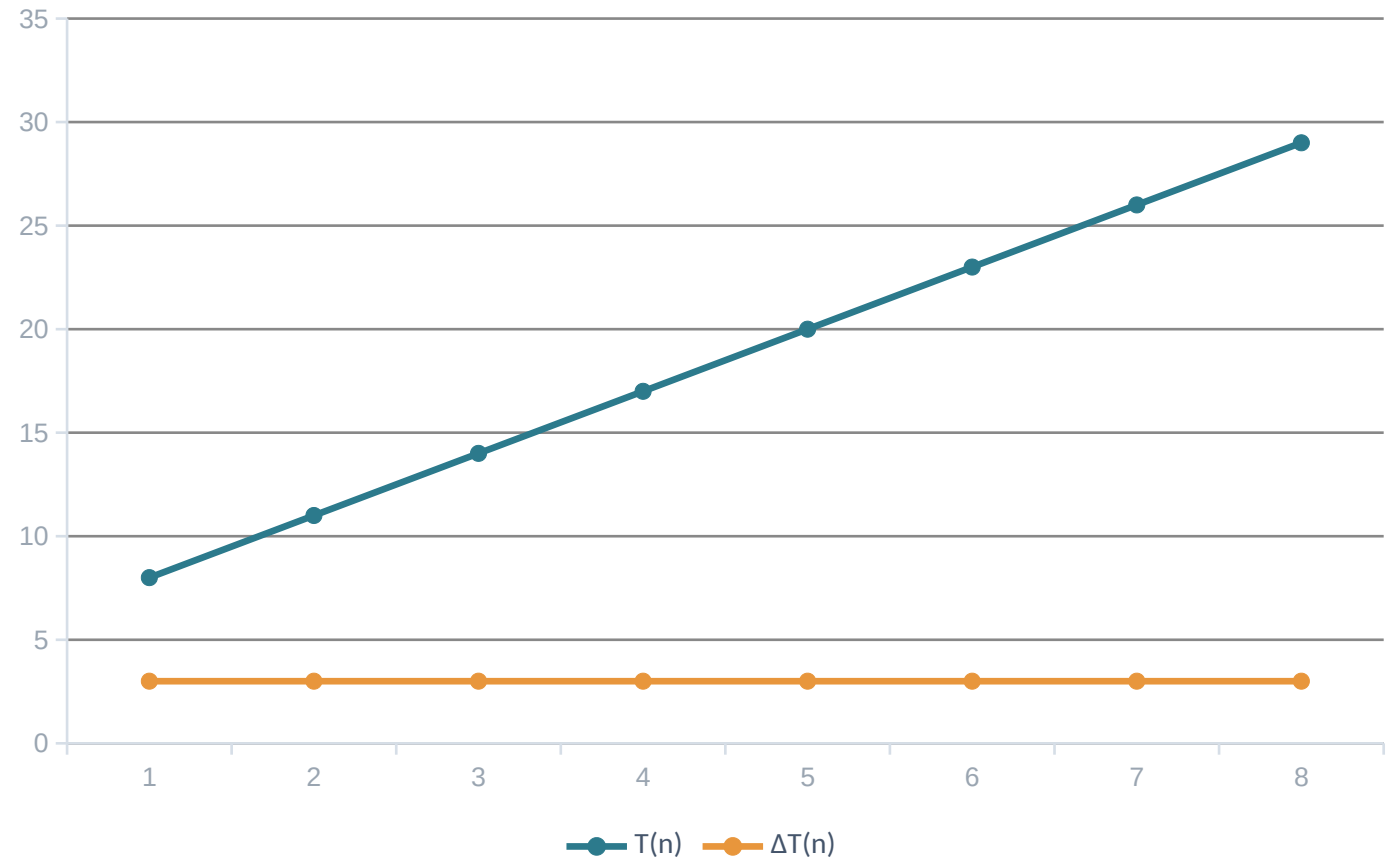
Corrida das curvas com barbante

Atividade desplugada — meça o crescimento com o corpo.

- 1 Material: barbante ou fita métrica, giz ou fita crepe no chão.
- 2 Marquem três retas de mesma origem no chão. Para $n = 5, 10$ e 15 , cortem pedaços de barbante proporcionais a n , a n^2 e a 2^n (em cm) e estiquem cada um na sua reta.
- 3 Com $n = 15$: o pedaço linear mede 15 cm, o quadrático 225 cm, e o exponencial passa de 300 metros — normalmente nem cabe na sala.
- 4 A distância entre os barbantes é a mesma distância que separa as curvas do gráfico — só que agora pode ser medida a passos.

Custo local estável

- Cada novo elemento acrescenta praticamente o mesmo bloco de trabalho.
- Exemplo: percorrer um vetor uma única vez.
- $T(n) = 3n + 5$ e $\Delta T(n) = 3$
- $\Delta T(n)$ é o custo marginal: quanto o total aumenta quando n cresce em 1 unidade — $\Delta T(n) = T(n) - T(n-1)$.
- Aqui esse aumento é sempre 3, não importa se n é 5 ou 50 — por isso o gráfico sobe em linha reta (ex.: $T(5) = 20$ e $T(4) = 17$, logo $\Delta T(5) = 20 - 17 = 3$).
- *Na prática: embalar caixas idênticas — cada caixa nova leva sempre os mesmos 3 minutos para fechar, não importa quantas já foram embaladas.*



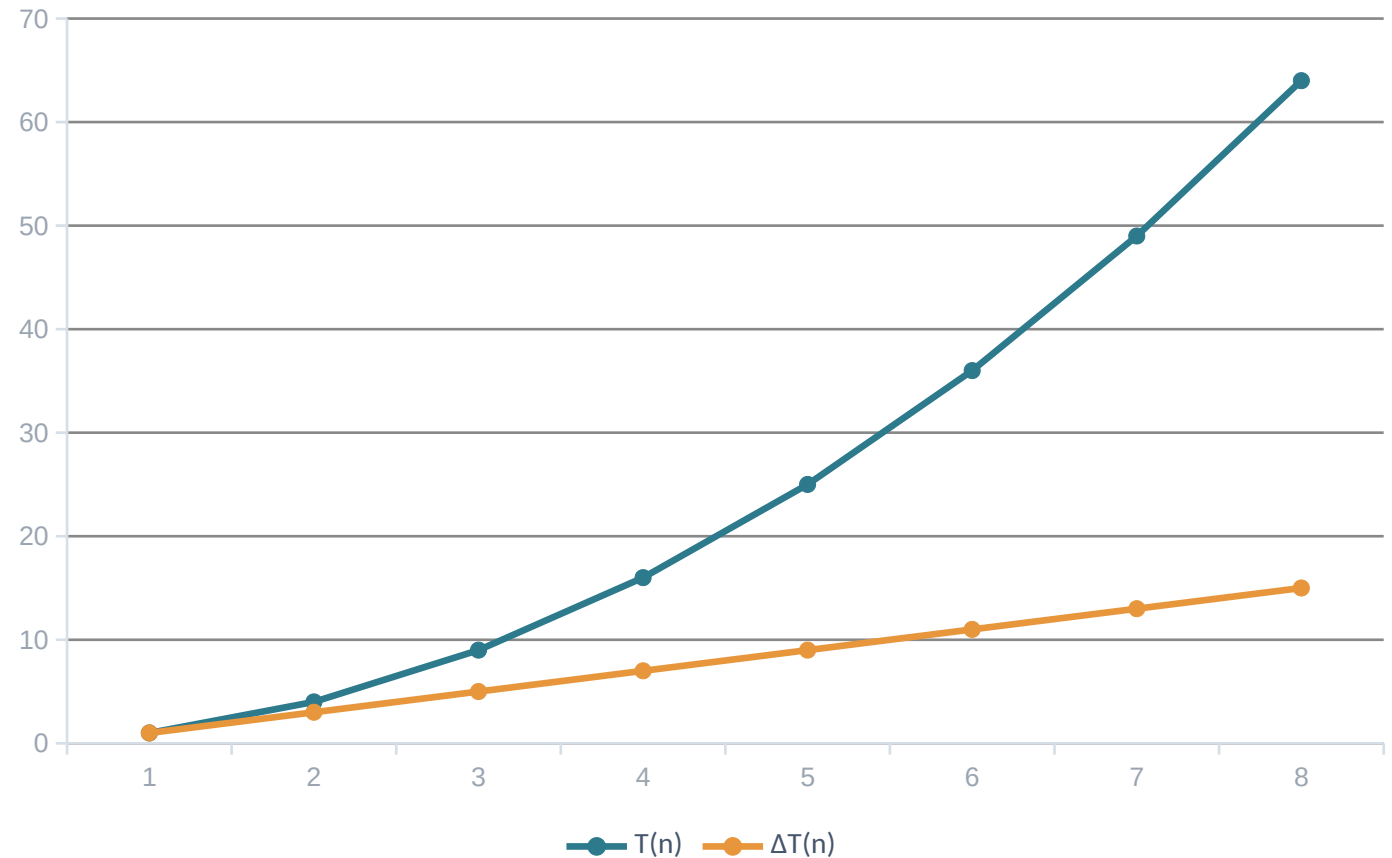
Linear em números

Colocando valores de n na fórmula $T(n) = 3n + 5$:

n	5	10	20
$T(n) = 3n + 5$	20	35	65
$\Delta T(n)$	3	3	3

Custo local crescente

- O trabalho adicional por novo elemento cresce com n .
- Por isso algoritmos quadráticos ficam rapidamente caros.
- $T(n) = n^2$ e $\Delta T(n) = 2n - 1$
- $\Delta T(n)$ é o custo marginal: quanto o total aumenta quando n cresce em 1 unidade — $\Delta T(n) = T(n) - T(n-1)$.
- Aqui esse aumento cresce junto com n ($2n - 1$) — por isso o gráfico curva para cima, cada vez mais rápido (ex.: $T(5) = 25$ e $T(4) = 16$, logo $\Delta T(5) = 25 - 16 = 9$).
- *Na prática: é o aperto de mãos da atividade — quem chega cumprimenta todo mundo que já está no grupo, então o custo extra cresce junto com n .*



Quadrático em números

Colocando valores de n na fórmula $T(n) = n^2$:

n	5	10	20
$T(n) = n^2$	25	100	400
$\Delta T(n) = 2n - 1$	9	19	39

Aperto de mãos, em números

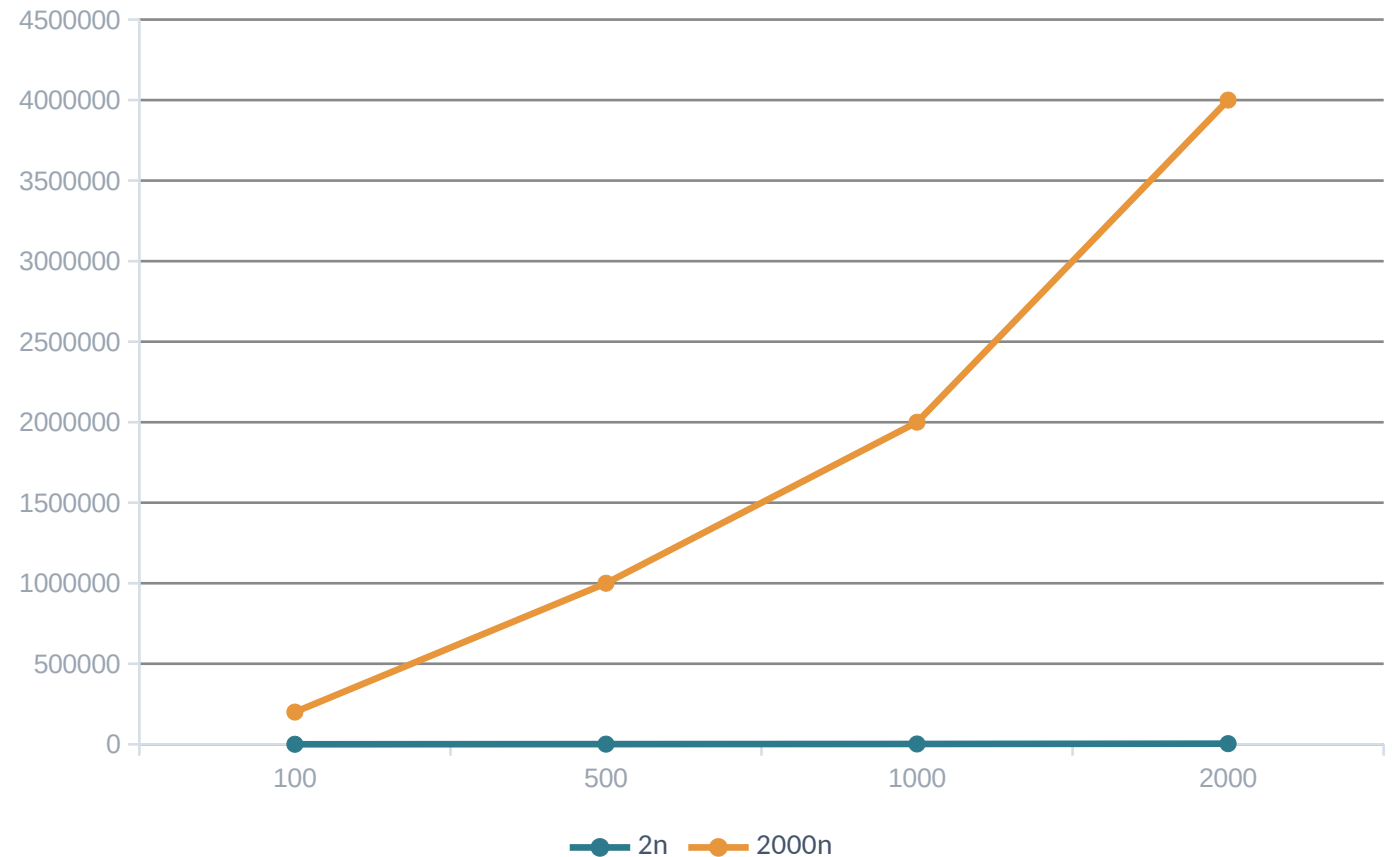
Aplicando a fórmula do aperto de mãos ($n \cdot (n-1)/2$) a um grupo real, com cada cumprimento levando 3 segundos:

n	5	10	20	30
T(n) — tempo total	30 seg	2 min 15 seg	9 min 30 seg	21 min 45 seg
$\Delta T(n)$ — quem chega por último	12 seg	27 seg	57 seg	87 seg

- n é o tamanho do grupo; $T(n)$ é o tempo total já gasto em cumprimentos até esse ponto; $\Delta T(n)$ é quanto a última pessoa a chegar gastou cumprimentando quem já estava lá.
- $T(n)$ cresce rápido porque soma todos os pares possíveis do grupo — com 30 pessoas (duas turmas juntas) já passa de 21 minutos só em cumprimentos.
- $\Delta T(n)$ não fica fixo: cada pessoa nova custa mais que a anterior, porque encontra um grupo maior pela frente — 12 segundos com 5 pessoas, 87 segundos com 30. É essa progressão que mostra, na prática, um custo marginal crescente.

Constantes escondidas ainda importam

- Ambas crescem proporcional a n .
- Mas a constante multiplicativa muda o custo concreto.
- Olhar para o crescimento avalia escalabilidade; $T(n)$ preserva eficiência aproximada.



Cada dobra dobra o tamanho

Atividade desplugada — sinta como o crescimento explosivo foge do controle rápido.

1

Material: uma folha de papel (quanto maior e mais fina, melhor).

2

Dobre a folha ao meio e conte: 1 dobra. Dobre de novo: 2 dobras. Continue até não conseguir mais.

3

Na prática, ninguém passa de 7 ou 8 dobras — mas se desse para continuar até 42 dobras, a pilha chegaria à Lua.

4

Cada dobra dobra a espessura anterior — é assim que um crescimento explosivo foge do controle rápido.

Quando cada passo dobra o anterior

- Na dobradura, o tamanho da pilha de papel dobra a cada passo — não soma, multiplica.
- É o mesmo tipo de crescimento da árvore de chamadas do Fibonacci ingênuo: cada chamada nova gera duas novas.
- Poucos passos bastam para o número ficar impossível de continuar na mão — ou no computador.

Fibonacci como função

- A definição é curta, mas a implementação recursiva ingênua gera uma árvore de chamadas.
- O problema não é “usar recursão”. O problema é recalcular os mesmos subproblemas.

$$F(0) = 0 \quad F(1) = 1 \quad F(n) = F(n-1) + F(n-2)$$

Árvore de chamadas com post-its

Atividade desplugada — construa a recomputação com as mãos.

1

Material: post-its (uma cor por valor de n) e um quadro ou mesa grande.

2

Em duplas, desenhem a árvore de chamadas de $\text{fib}(5)$: um post-it para cada chamada — $\text{Fib}(5)$, $\text{Fib}(4)$, $\text{Fib}(3)$...

3

Toda vez que um valor de n já apareceu antes, cole o novo post-it em cima do anterior, não ao lado.

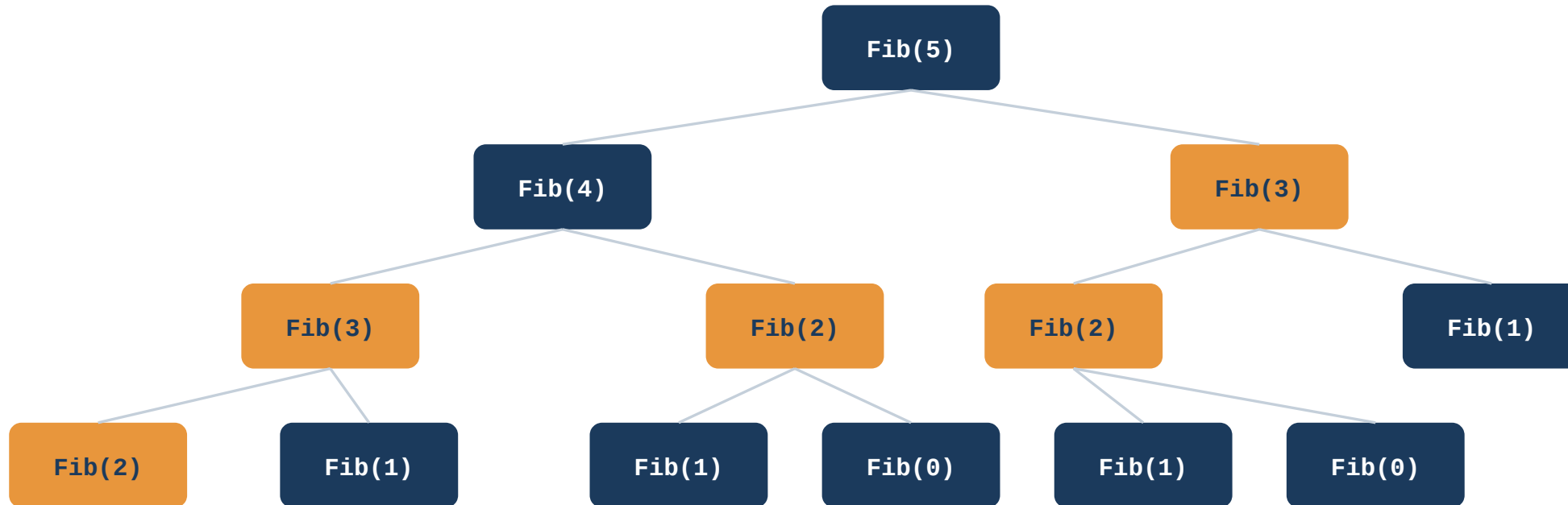
4

A pilha de post-its sobrepostos mostra fisicamente o quanto foi recomputado — e antecipa a ideia de memoização.

Fibonacci recursivo ingênuo

- Há poucas linhas de código, mas muitas chamadas repetidas.
- Linhas de código não medem complexidade.
- A estrutura de chamadas é o objeto da análise.

Árvore de chamadas



Fib(3) e Fib(2) aparecem mais de uma vez. Isso é recomputação.

Custo do Fibonacci ingênuo

- Crescimento aproximado: $T(n) = T(n-1) + T(n-2) + 1$
- A soma das duas chamadas gera uma expansão em árvore.
- $\varphi = (1+\sqrt{5})/2 \approx 1,618$

$$T(n) = T(n-1) + T(n-2) + 1 \quad \rightarrow \quad \text{quase dobra a cada chamada}$$

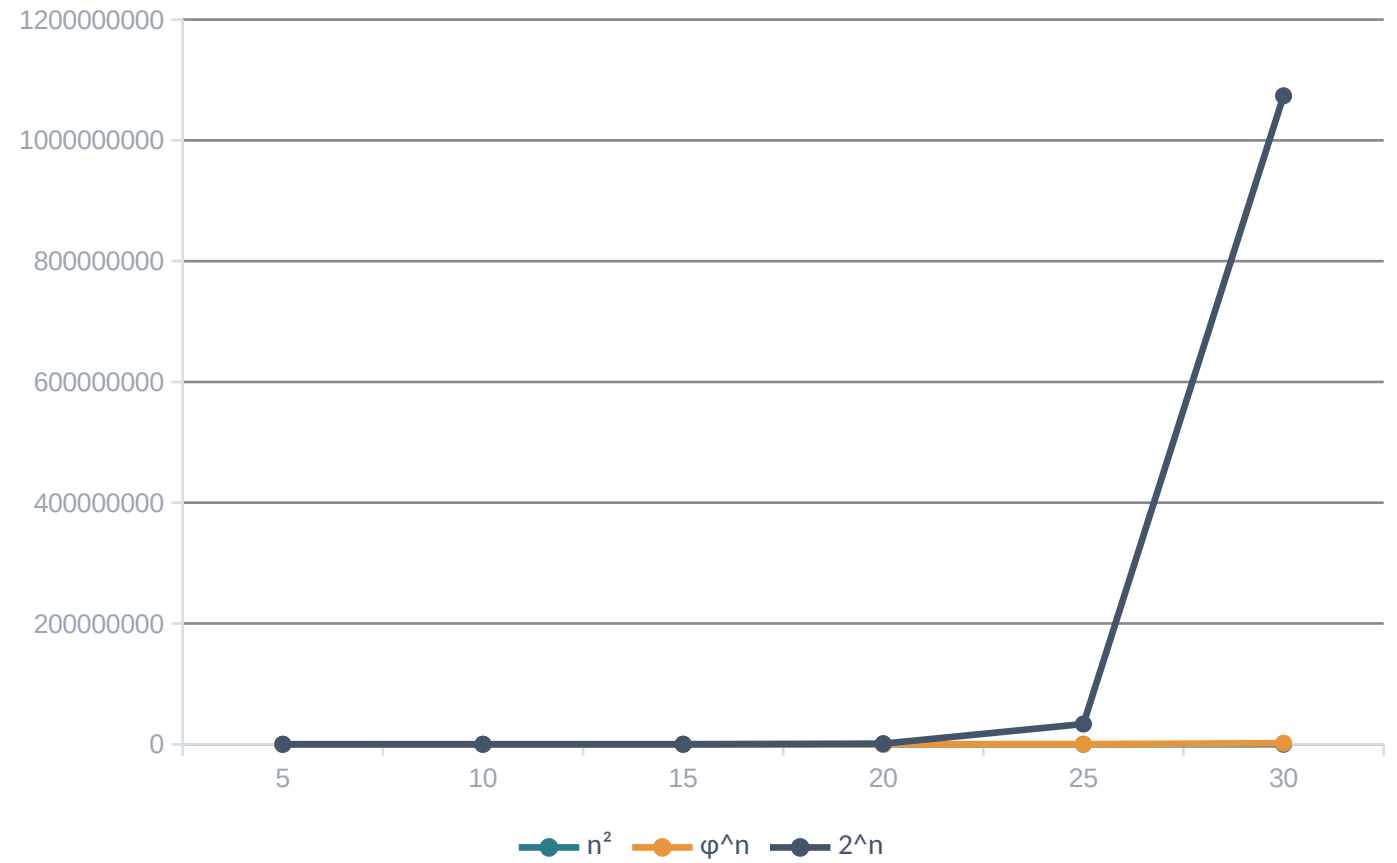
A recorrência em números

Calculando $T(n) = T(n-1) + T(n-2) + 1$ passo a passo, a partir de $T(0) = T(1) = 1$:

n	0	1	2	3	4	5
$T(n)$	1	1	3	5	9	15

Polinomial versus exponencial

- Em n^2 , a variável está na base.
- Em φ^n , a variável está no expoente.
- Essa diferença explica a explosão.



Memoização na lousa

Atividade desplugada — sinta a diferença entre recalcular e consultar.

1

Material: um quadro com uma tabela vazia de $F(0)$ a $F(10)$.

2

Um aluno é o “processador”: para calcular $F(n)$, ele deve olhar a tabela antes de fazer qualquer conta.

3

Se a casa já está preenchida, ele só aponta para o valor — não recalcula. Se está vazia, calcula, escreve e segue.

4

Cronometre as duas versões — recalculando tudo $\times$ consultando a tabela — a diferença de tempo é a diferença entre recursão ingênua e programação dinâmica.

Programação dinâmica

Sem memória

- O mesmo subproblema é resolvido várias vezes.

Com memoização

- Cada estado $F(i)$ é calculado uma única vez e armazenado.
- Custo: proporcional a n

Fibonacci iterativo

- O algoritmo percorre os estados de 0 até n .
- Tempo: proporcional a n
- Memória: praticamente fixa
- Não há recomputação.

Atividade em sala

Antes de conferir a resposta, peça que os alunos respondam:

- 1 Qual é a ação fundamental?
- 2 Quantas vezes ela ocorre?
- 3 A sequência gera constante aditiva ou multiplicativa?
- 4 O IF muda a ordem ou apenas o caminho?
- 5 Se dobrar n , o trabalho dobra, quadruplica ou cresce mais?

Como o trabalho cresce importa mais que o código

01

Como o trabalho cresce importa mais que o código

O que importa não é quantas linhas o código tem, mas quantas vezes cada passo se repete conforme os dados crescem.

02

Laços moldam o crescimento

Sequências calibram constantes; laços aninhados multiplicam o crescimento — é aí que nascem os crescimentos quadrático e outros ainda mais rápidos.

03

Programação dinâmica elimina recomputação

Guardar resultados de subproblemas troca uma explosão exponencial, como no Fibonacci ingênuo, por custo linear.



A SEGUIR — AULA 15 de 15

Encerramento e Discussão

Referências

RAABE, André; ZORZO, Avelino F.; BLIKSTEIN, Paulo. Computação na Educação Básica: Fundamentos e Experiências. Porto Alegre: Penso, 2020. Ebook. ISBN 9786581334048.

RIBEIRO, João Araújo. Introdução à Programação e aos algoritmos. Rio de Janeiro: LTC, 2019. ISBN 978-85-216-3640-3.